Università degli Studi di Messina

CdL in Scienze Informatiche
Basi di Dati - Modulo NoSQL
A.A. 2024/2025

Benchmark delle Prestazioni tra Oracle (RDBMS) e Neo4j (Grafo) nell'Identificazione di Frodi IVA

Sandi Russo, 553675

24 gennaio 2026

Sommario

Questo studio presenta un'analisi comparativa delle prestazioni tra un database relazionale, Oracle, e un database a grafo, Neo4j, nel contesto dell'identificazione di frodi IVA. Attraverso la generazione di un dataset sintetico su larga scala, sono state confrontate le performance dei due sistemi su quattro query archetipiche, progettate per simulare sia ricerche basate su proprietà aggregate sia l'analisi di pattern relazionali complessi come le triangolazioni. I risultati dimostrano la superiorità di Oracle nelle operazioni set-based a bassa profondità e il dominio di Neo4j nell'attraversamento di percorsi complessi. La relazione conclude che, sebbene nessuna tecnologia sia universalmente superiore, un'architettura ibrida che sfrutta i punti di forza di entrambi i paradigmi rappresenta la soluzione più efficace per un'analisi delle frodi completa in un contesto enterprise.

Indice

1 Problema Affrontato: le Frodi IVA **2**
1.1 Generazione e Scala del Dataset 2

2 Le Soluzioni DBMS a Confronto **3**
2.1 Oracle e il Paradigma Relazionale 3
2.1.1 Architettura e Punti di Forza 3
2.1.2 Limiti nel Contesto delle Frodi 3
2.2 Neo4j e il Paradigma a Grafo 3
2.2.1 Architettura e Punti di Forza 4
2.2.2 Limiti nel Contesto delle Frodi 4

3 Progettazione e Modellazione dei Dati **5**
3.1 Modello Relazionale (Oracle) 5
3.1.1 Schema Fisico 5
3.1.2 Definizione delle Tabelle (DDL) 5
3.2 Modello a Grafo (Neo4j) 6
3.2.1 Struttura del Grafo 6
3.2.2 Visualizzazione Grafica 7

4 Metodologia Sperimentale **8**
4.1 Ambiente di Test 8

5 Analisi delle Query di Benchmark **9**
5.1 Query 1: Elenco Aziende in Nazioni a Rischio 9
5.2 Query 2: Individuazione di Triangolazioni Semplici 11
5.3 Query 3: Conteggio Transazioni Internazionali 13
5.4 Query 4: Ricerca Flussi tra UE e Paradisi Fiscali 15

6 Conclusioni Finali **17**

1 Problema Affrontato: le Frodi IVA

Le frodi sull'Imposta sul Valore Aggiunto (IVA), e in particolare le cosiddette *frodi carosello*, costituiscono una delle maggiori fonti di perdita di gettito fiscale per gli Stati membri dell'Unione Europea e per le economie globali. Tali schemi fraudolenti si basano su catene di transazioni, spesso transnazionali, architettate con il solo scopo di evadere il pagamento dell'IVA e/o di ottenere indebiti rimborsi.

Le caratteristiche principali di queste frodi sono:

- **Complessità Relazionale:** Il cuore della frode non risiede in una singola transazione, ma in una rete complessa di rapporti tra più entità (aziende, persone, banche). L'individuazione richiede di analizzare non solo i dati, ma soprattutto le **connessioni** tra di essi.
- **Società Cartiera (*Missing Trader*):** Vengono create società fittizie che acquistano beni o servizi, addebitano l'IVA ai loro clienti, ma scompaiono senza versarla allo Stato.
- **Triangolazioni e Flussi Anomali:** Il denaro e le merci seguono percorsi non lineari, spesso passando per intermediari in paesi con regimi fiscali differenti o paradisi fiscali, per offuscare l'origine e la destinazione dei flussi.
- **Volume dei Dati:** Le autorità fiscali devono analizzare milioni di transazioni, anagrafiche e bilanci, rendendo l'analisi manuale impraticabile.

Questo studio si propone di valutare l'efficacia di due tecnologie di database antitetiche per affrontare questa sfida: **Oracle**, un database relazionale (RDBMS) maturo e leader di mercato, e **Neo4j**, il più popolare database a grafo nativo. L'obiettivo è determinare quale paradigma si presti meglio all'analisi di pattern di frode complessi, confrontando le loro prestazioni su un vasto dataset sintetico che simula uno scenario realistico.

1.1 Generazione e Scala del Dataset

Per condurre un'analisi rigorosa, è stato generato un dataset di base (profilo 100%) con le seguenti cardinalità:

- **Nazioni:** 190
- **Banche:** 500
- **Aziende:** 70.000
- **Persone:** 150.000
- **Carte di Credito:** 100.000
- **Transazioni B2B:** 500.000

Da questo dataset sono stati derivati tre sottoinsiemi di dimensioni decrescenti (75%, 50%, 25%) per valutare come le prestazioni dei due DBMS scalano al variare del volume dei dati.

2 Le Soluzioni DBMS a Confronto

La scelta del database è un fattore critico che influenza non solo le prestazioni, ma anche la facilità di modellazione del problema e l'espressività delle query.

2.1 Oracle e il Paradigma Relazionale

Oracle Database è l'archetipo del sistema di gestione di database relazionale (RDBMS). Il suo modello, basato sull'algebra relazionale, organizza i dati in **tabelle** bidimensionali composte da righe e colonne. Le relazioni tra le diverse tabelle sono logicamente definite tramite **chiavi primarie ed esterne**.

2.1.1 Architettura e Punti di Forza

- **Consistenza e Affidabilità (ACID):** Oracle è costruito attorno alle proprietà ACID (Atomicità, Consistenza, Isolamento, Durabilità). Questo garantisce che ogni transazione sia eseguita in modo affidabile, rendendolo la scelta d'elezione per sistemi critici come quelli finanziari e bancari.
- **Potenza nelle Operazioni Set-Based:** Il linguaggio SQL e il motore di Oracle sono eccezionalmente ottimizzati per operazioni su insiemi di dati (*set-based*). Query che richiedono di scansionare, filtrare, aggregare (`SUM`, `COUNT`) e ordinare grandi quantità di righe sono il suo punto di forza.
- **Maturità ed Ecosistema:** Decenni di sviluppo hanno prodotto un sistema stabile, sicuro e con un ecosistema di strumenti, documentazione e competenze professionali senza pari.
- **Ottimizzatore di Query Sofisticato:** L'ottimizzatore di Oracle è uno dei più avanzati al mondo, capace di analizzare una query SQL e di scegliere il piano di esecuzione più efficiente (es. quale indice usare, in che ordine eseguire i join).

2.1.2 Limiti nel Contesto delle Frodi

Il principale svantaggio del modello relazionale emerge quando si analizzano relazioni complesse e percorsi lunghi. L'identificazione di una triangolazione `A -> B -> C` richiede in SQL una serie di operazioni di `JOIN`. Ogni `JOIN` è computazionalmente costoso, e il costo cresce esponenzialmente con la profondità della ricerca, poiché il database deve manipolare insiemi di risultati intermedi sempre più grandi.

2.2 Neo4j e il Paradigma a Grafo

Neo4j è un database a grafo **nativo**. A differenza dei sistemi relazionali, non modella le connessioni tramite puntatori logici (chiavi esterne), ma le tratta come elementi di prima classe. Il suo modello si fonda su:

- **Nodi:** Rappresentano le entità (es. un'Azienda, una Nazione).
- **Relazioni:** Connessioni dirette, tipizzate e direzionate tra i nodi (es. `[:REGISTRATA_IN]`).

- **Proprietà:** Coppie chiave-valore per arricchire nodi e relazioni con metadati.

2.2.1 Architettura e Punti di Forza

- **Prestazioni nell'Attraversamento:** La sua caratteristica distintiva è l'*Index-Free Adjacency*. Ogni nodo memorizza un puntatore fisico diretto alle sue relazioni e ai nodi adiacenti. Quando si esegue una query che segue un percorso (es. '(Azienda)->(Transazione)->(Azienda)'), Neo4j attraversa questi puntatori in memoria. La performance di questa operazione dipende solo dal numero di nodi attraversati nel percorso, non dalla dimensione totale del database. Questo rende le query su percorsi complessi e profondi estremamente veloci.

- **Modellazione Intuitiva:** Il modello a grafo, spesso rappresentato su una lavagna come "bolle e frecce", si traduce quasi direttamente nello schema del database. Questo rende la modellazione di domini complessi come le frodi molto più naturale e comprensibile.

- **Linguaggio Cypher:** Cypher è un linguaggio dichiarativo progettato per esprimere pattern di grafo in modo ASCII-art. Una query come `MATCH (a1)-[]->(a2)-[]->(a3)` è sia leggibile che potente.

2.2.2 Limiti nel Contesto delle Frodi

Neo4j è meno efficiente quando deve rispondere a domande che non partono da un punto specifico del grafo ma richiedono di scansionare e aggregare proprietà su tutti i nodi di un certo tipo (es. "qual è l'importo medio di tutte le transazioni?"). In questi casi, un RDBMS che può eseguire una scansione tabellare ottimizzata è spesso più veloce.

3 Progettazione e Modellazione dei Dati

3.1 Modello Relazionale (Oracle)

In Oracle, lo schema è stato definito tramite tabelle normalizzate per garantire l'integrità dei dati ed evitare ridondanze. Le relazioni sono implementate tramite vincoli di Fk

3.1.1 Schema Fisico

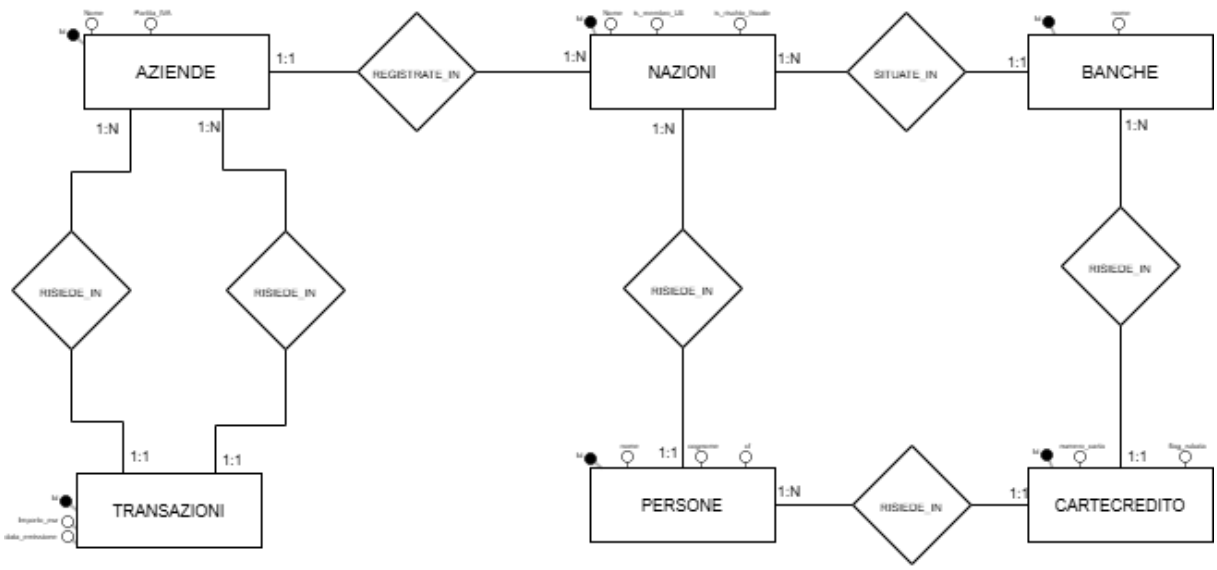

Figura 1: Schema Entità-Relazione implementato in Oracle.

3.1.2 Definizione delle Tabelle (DDL)

```
CREATE TABLE Nazioni (
    id_nazione NUMBER PRIMARY KEY,
    nome VARCHAR2(100),
    codice_iso_alpha2 CHAR(2) UNIQUE NOT NULL,
    is_membro_ue CHAR(1) CHECK (is_membro_ue IN ('S', 'N')),
    is_rischio_fiscale CHAR(1) CHECK (is_rischio_fiscale IN ('S', 'N'))
);

CREATE TABLE Banche (
    id_banca NUMBER PRIMARY KEY,
    nome VARCHAR2(100),
    id_nazione NUMBER REFERENCES Nazioni(id_nazione)
);

CREATE TABLE Aziende (
    id_azienda NUMBER PRIMARY KEY,
    nome VARCHAR2(100),
    partita_iva VARCHAR2(20) UNIQUE,
    id_nazione NUMBER REFERENCES Nazioni(id_nazione)
);
```

```sql

CREATE TABLE Persone (
    id_persona NUMBER PRIMARY KEY,
    nome VARCHAR2(100),
    cognome VARCHAR2(100),
    codice_fiscale VARCHAR2(20) UNIQUE,
    id_nazione_residenza NUMBER REFERENCES Nazioni(id_nazione)
);

CREATE TABLE CarteCredito (
    id_carta NUMBER PRIMARY KEY,
    numero_carta VARCHAR2(20) UNIQUE,
    data_scadenza DATE,
    id_persona NUMBER REFERENCES Persone(id_persona),
    id_banca NUMBER REFERENCES Banche(id_banca),
    limite_spesa_eur NUMBER(10,2),
    flag_attiva CHAR(1) CHECK (flag_attiva IN ('S', 'N')),
    flag_rubata_smarrita CHAR(1) CHECK (flag_rubata_smarrita IN ('S', 'N'))
);

CREATE TABLE Transazioni (
    id_transazione NUMBER PRIMARY KEY,
    id_azienda_emittente NUMBER REFERENCES Aziende(id_azienda),
    id_azienda_destinataria NUMBER REFERENCES Aziende(id_azienda),
    data_emissione DATE,
    importo_eur NUMBER(10,2),
    aliquota_iva NUMBER(5,2),
    CONSTRAINT chk_trans_aziende_diverse CHECK (id_azienda_emittente <> id_azienda_destinataria)
);
```

Listing 1: DDL per le tabelle principali in Oracle

3.2 Modello a Grafo (Neo4j)

In Neo4j, il modello si concentra sulla rappresentazione esplicita delle connessioni. Le entità diventano nodi e i legami diventano relazioni.

3.2.1 Struttura del Grafo

- **Nodi (Labels):** `:Azienda`, `:Nazione`, `:TransazioneB2B`, `:Persona`, `:CartaCredito`, `:Banca`.
- **Relazioni (Types):**
  - `(Azienda)-[:REGISTRATA_IN]->(Nazione)`
  - `(TransazioneB2B)-[:EMESSA_DA_AZIENDA]->(Azienda)`
  - `(TransazioneB2B)-[:DESTINATA_AD_AZIENDA]->(Azienda)`
  - `(Persona)-[:RISIEDE_IN]->(Nazione)`
  - `(CartaCredito)-[:POSSEDUTA_DA]->(Persona)`

- `(CartaCredito)-[:EMESSA_DA_BANCA]->(Banca)`
- `(Banca)-[:SITUATA_IN]->(Nazione)`

Questo approccio elimina la necessità dei `JOIN` computazionali. Per trovare le transazioni di un'azienda, si parte dal nodo `Azienda` e si seguono le relazioni.

3.2.2 Visualizzazione Grafica

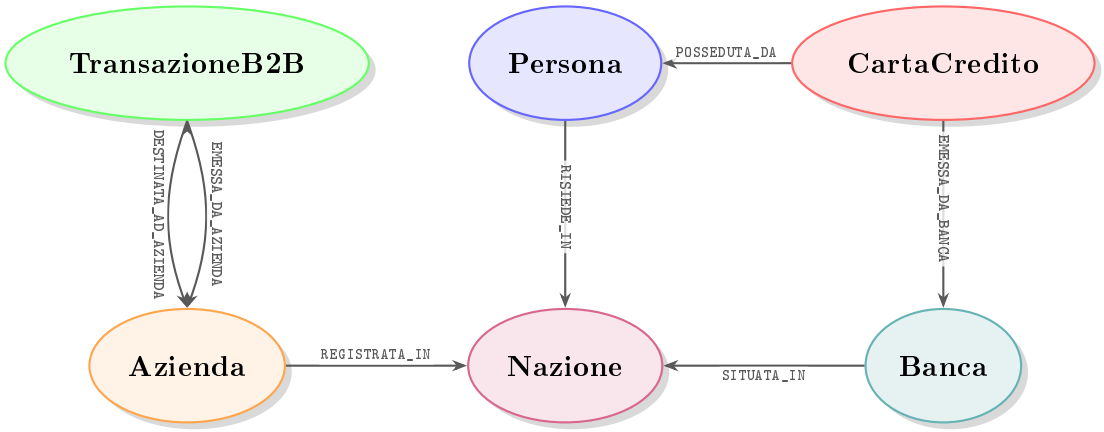

Figura 2: Schema del grafo Neo4j con le principali entità e relazioni.

4 Metodologia Sperimentale

Tutti i test sono stati eseguiti sulla medesima macchina per garantire la coerenza dei risultati.

4.1 Ambiente di Test

Tutti i test sono stati eseguiti sulla medesima macchina per garantire la coerenza dei risultati.

- **CPU:** AMD RYZEN 7-5700U
- **RAM:** 16 GB
- **Storage:** SSD NVMe da 1024 GB
- **Sistema Operativo:** Windows 11 Pro / Ubuntu 22.04 LTS (su Docker)
- **Software:** Oracle Database 21c, Neo4j Community Edition
- **Librerie Chiave di Python:** oracledb, neo4j
- **Software:** Python per l'orchestrazione dei test, librerie 'oracledb' per Oracle e 'neo4j' per Neo4j.
- **Procedura di Benchmark:** Per ogni coppia (query, dataset), sono state eseguite 50 interrogazioni.
  - **Esecuzione a Freddo (Cold Run):** Tempo della prima esecuzione, a cache vuota.
  - **Esecuzione a Caldo (Hot Run):** Media aritmetica delle 49 esecuzioni successive, per valutare le prestazioni a regime, con le cache popolate.

5 Analisi delle Query di Benchmark

5.1 Query 1: Elenco Aziende in Nazioni a Rischio

Obiettivo: Estrarre un semplice elenco di aziende che si trovano in nazioni considerate a rischio fiscale e non appartenenti all'UE. Questa è una query di filtraggio con un singolo `JOIN`, tipica del reporting di base.

```
SELECT a.nome AS nome_azienda, n.nome AS nazione
FROM Aziende a
JOIN Nazioni n ON a.id_nazione = n.id_nazione
WHERE n.is_rischio_fiscale = 'S' AND n.is_membro_ue = 'N'
```

Listing 2: Q1 in Oracle SQL

```
MATCH (a:Azienda)-[:REGISTRATA_IN]->(n:Nazione {is_rischio_fiscale: 'S',
    is_membro_ue: 'N'})
RETURN a.nome AS nome_azienda, n.nome AS nazione
```

Listing 3: Q1 in Neo4j Cypher

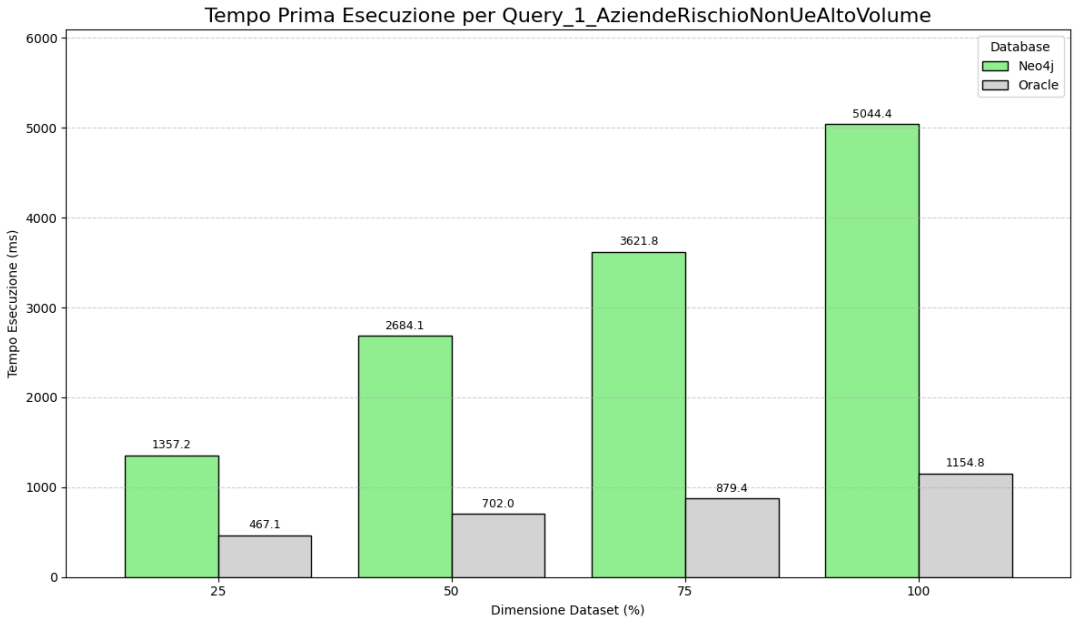

Figura 3: Q1 - Tempi di esecuzione a freddo.

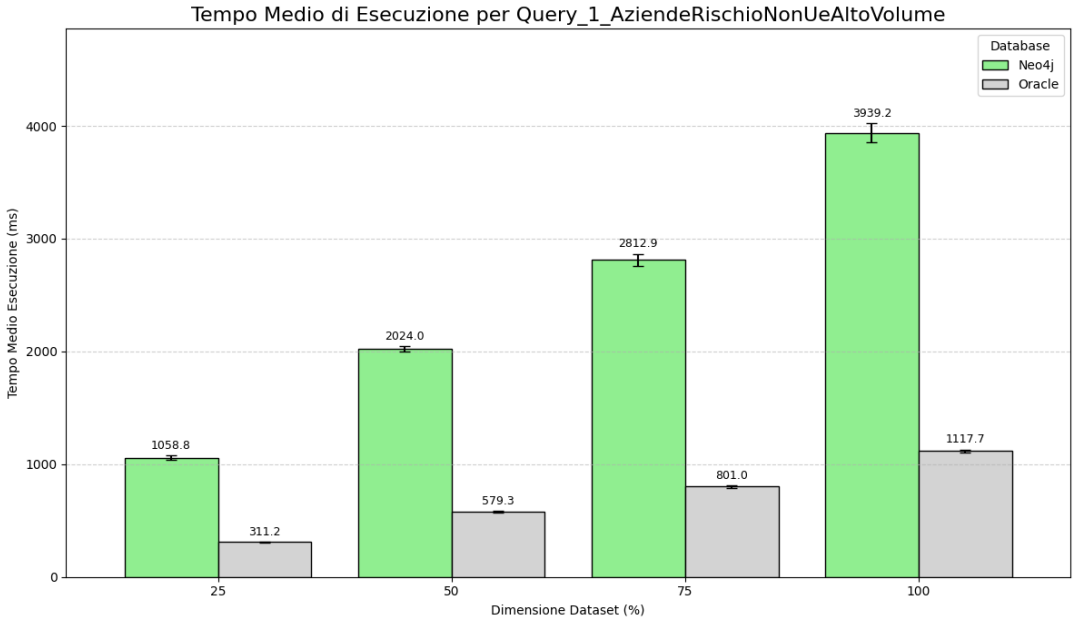

Figura 4: Q1 - Tempi medi di esecuzione a caldo.

Analisi Q1: Oracle è vincitore. Questa query rappresenta un caso d'uso classico per un RDBMS. L'operazione consiste nel filtrare la tabella 'Nazioni' (un'operazione molto veloce, specialmente con un indice) e poi eseguire un `JOIN` con la tabella 'Aziende'. Il motore SQL di Oracle è estremamente ottimizzato per queste operazioni *set-based* su percorsi brevi. Neo4j è comunque efficiente, ma l'overhead della sua architettura per una query così semplice non gli consente di superare la specializzazione di Oracle.

5.2 Query 2: Individuazione di Triangolazioni Semplici

Obiettivo: Identificare percorsi di lunghezza 2 della forma `(Azienda1) -> (Azienda2) -> (Azienda3)`, che rappresentano la base per la ricerca di frodi carosello.

```sql
SELECT DISTINCT t1.id_azienda_emittente, t2.id_azienda_destinataria
FROM Transazioni t1
JOIN Transazioni t2 ON t1.id_azienda_destinataria = t2.
    id_azienda_emittente
WHERE t1.id_azienda_emittente <> t2.id_azienda_destinataria
```

Listing 4: Q2 in Oracle SQL

```cypher
MATCH (a1:Azienda)<-[:EMESSA_DA_AZIENDA]-(:TransazioneB2B)-[:
    DESTINATA_AD_AZIENDA]->(:Azienda)<-[:EMESSA_DA_AZIENDA]-(:
    TransazioneB2B)-[:DESTINATA_AD_AZIENDA]->(a3:Azienda)
WHERE a1 <> a3
RETURN DISTINCT a1.nome, a3.nome
```

Listing 5: Q2 in Neo4j Cypher

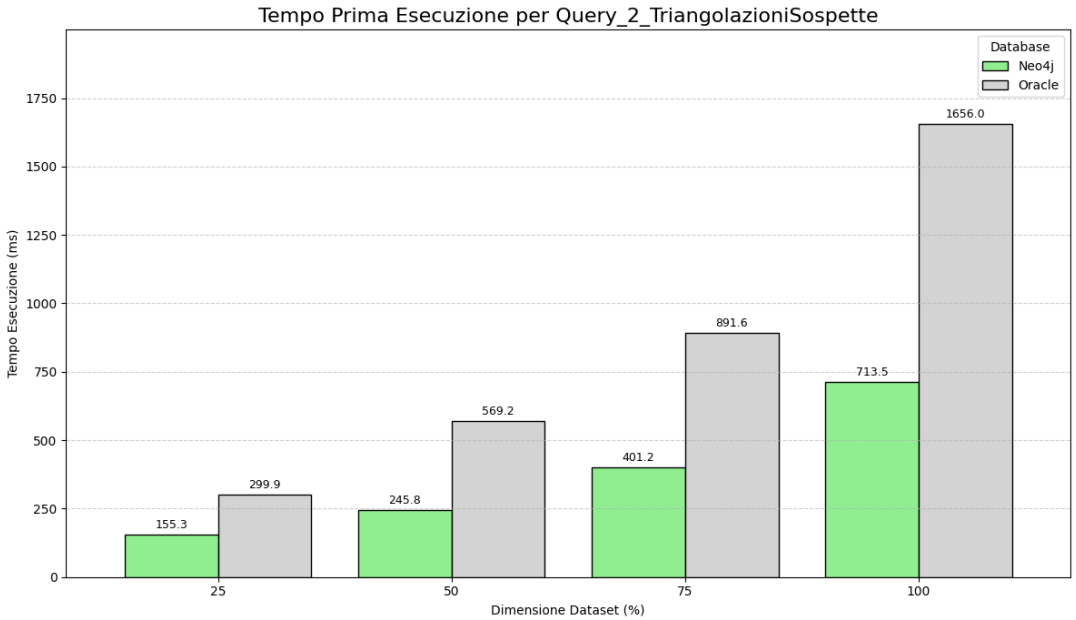

Figura 5: Q2 - Tempi di esecuzione a freddo.

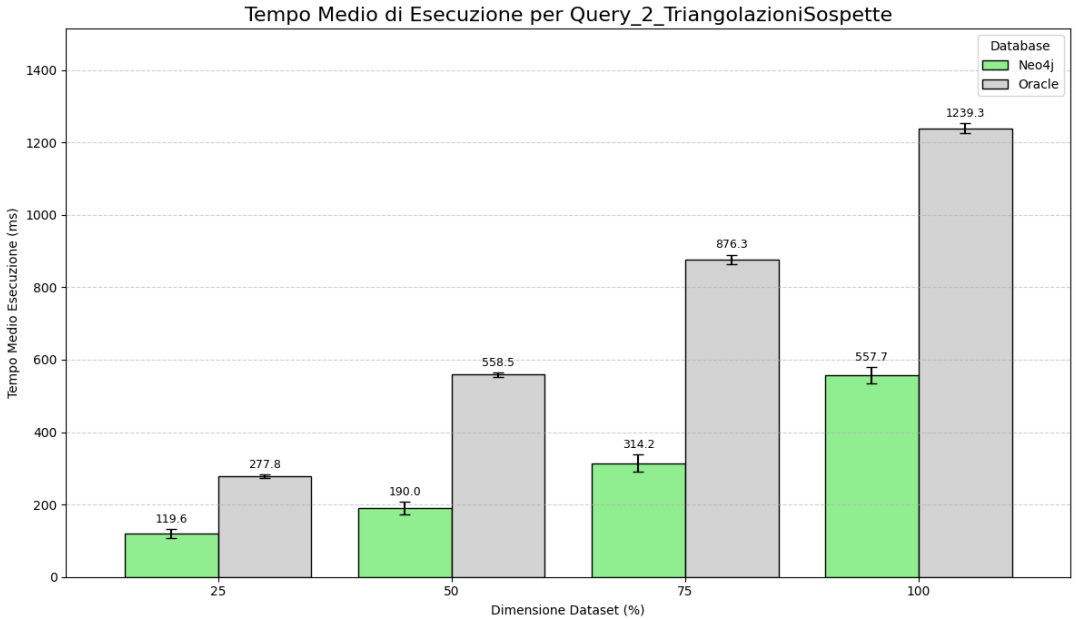

Figura 6: Q2 - Tempi medi di esecuzione a caldo.

Analisi Q2: Neo4j domina nettamente. Questo è il caso d'uso per eccellenza di un database a grafo. La query SQL richiede un **self-join** sulla tabella 'Transazioni', che è potenzialmente enorme. Questa operazione è computazionalmente molto costosa per un RDBMS. Al contrario, la query Cypher descrive un pattern di attraversamento che il motore di Neo4j esegue in modo nativo ed efficiente grazie all'*Index-Free Adjacency*, seguendo semplicemente i puntatori in memoria.

5.3 Query 3: Conteggio Transazioni Internazionali

Obiettivo: Trovare tutte le transazioni in cui l'azienda emittente e quella destinataria sono registrate in nazioni diverse. Questa query richiede di accedere a dati di entità collegate, ma non di esplorare percorsi profondi.

```
SELECT t.id_transazione
FROM Transazioni t
JOIN Aziende ae ON t.id_azienda_emittente = ae.id_azienda
JOIN Aziende ad ON t.id_azienda_destinataria = ad.id_azienda
WHERE ae.id_nazione <> ad.id_nazione
```

Listing 6: Q3 in Oracle SQL

```
MATCH (ae:Azienda)-[:REGISTRATA_IN]->(ne:Nazione)
MATCH (ad:Azienda)-[:REGISTRATA_IN]->(nd:Nazione)
WHERE ne <> nd
MATCH (ae)<-[:EMESSA_DA_AZIENDA]-(t:TransazioneB2B)-[:
    DESTINATA_AD_AZIENDA]->(ad)
RETURN t.id_transazione
```

Listing 7: Q3 in Neo4j Cypher

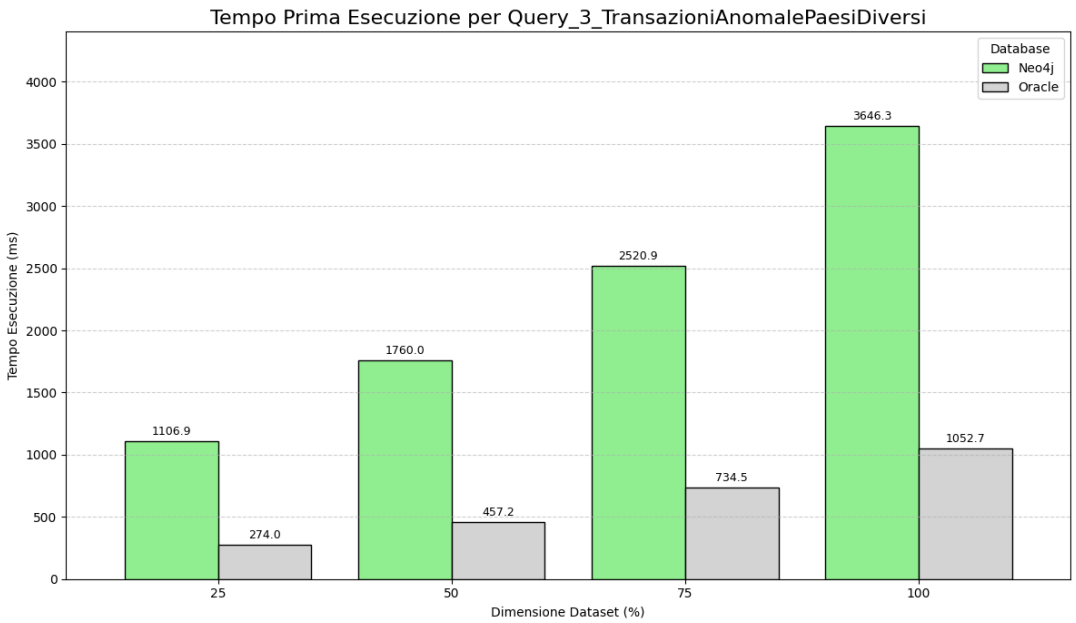

Figura 7: Q3 - Tempi di esecuzione a freddo.

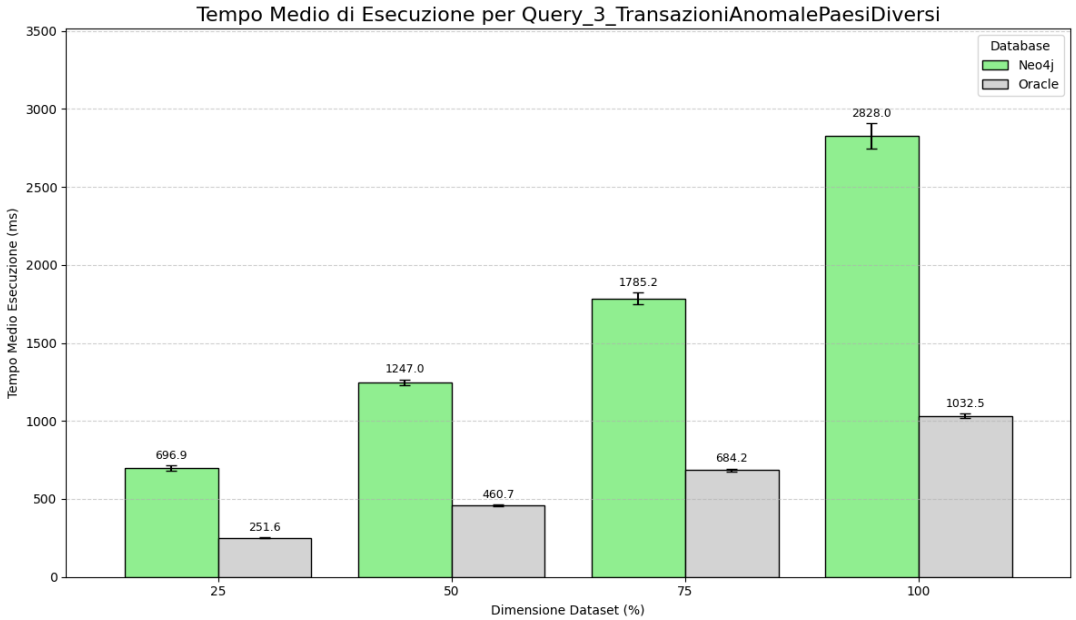

Figura 8: Q3 - Tempi medi di esecuzione a caldo.

Analisi Q3: Oracle risulta più performante. Similmente alla Query 1, il carico di lavoro principale è una scansione della grande tabella 'Transazioni', seguita da `JOIN` per verificare una condizione sulle entità collegate. Sebbene richieda due `JOIN` alla tabella 'Aziende', la profondità di ricerca rimane bassa. L'ottimizzatore di Oracle è in grado di gestire questo scenario in modo molto efficiente, mentre la query Cypher, che deve risolvere più pattern, risulta leggermente più complessa per il motore a grafo.

5.4 Query 4: Ricerca Flussi tra UE e Paradisi Fiscali

Obiettivo: Individuare catene di transazioni che partono da un'azienda in un paese UE e terminano in un'azienda situata in un paese a rischio fiscale, un chiaro indicatore di potenziale evasione.

```sql
SELECT DISTINCT a1.nome, a3.nome
FROM Transazioni t1
JOIN Aziende a1 ON t1.id_azienda_emittente = a1.id_azienda
JOIN Nazioni n1 ON a1.id_nazione = n1.id_nazione
JOIN Transazioni t2 ON t1.id_azienda_destinataria = t2.
    id_azienda_emittente
JOIN Aziende a3 ON t2.id_azienda_destinataria = a3.id_azienda
JOIN Nazioni n3 ON a3.id_nazione = n3.id_nazione
WHERE n1.is_membro_ue = 'S' AND n3.is_rischio_fiscale = 'S'
```

Listing 8: Q4 in Oracle SQL

```cpp
MATCH (a1:Azienda)-[:REGISTRATA_IN]->(:Nazione {is_membro_ue: 'S'}),
      (a1)<-[:EMESSA_DA_AZIENDA]-(:TransazioneB2B)-[:
    DESTINATA_AD_AZIENDA]->(:Azienda)<-[:EMESSA_DA_AZIENDA]-(:
    TransazioneB2B)-[:DESTINATA_AD_AZIENDA]->(a3:Azienda),
      (a3)-[:REGISTRATA_IN]->(:Nazione {is_rischio_fiscale: 'S'})
RETURN DISTINCT a1.nome, a3.nome
```

Listing 9: Q4 in Neo4j Cypher

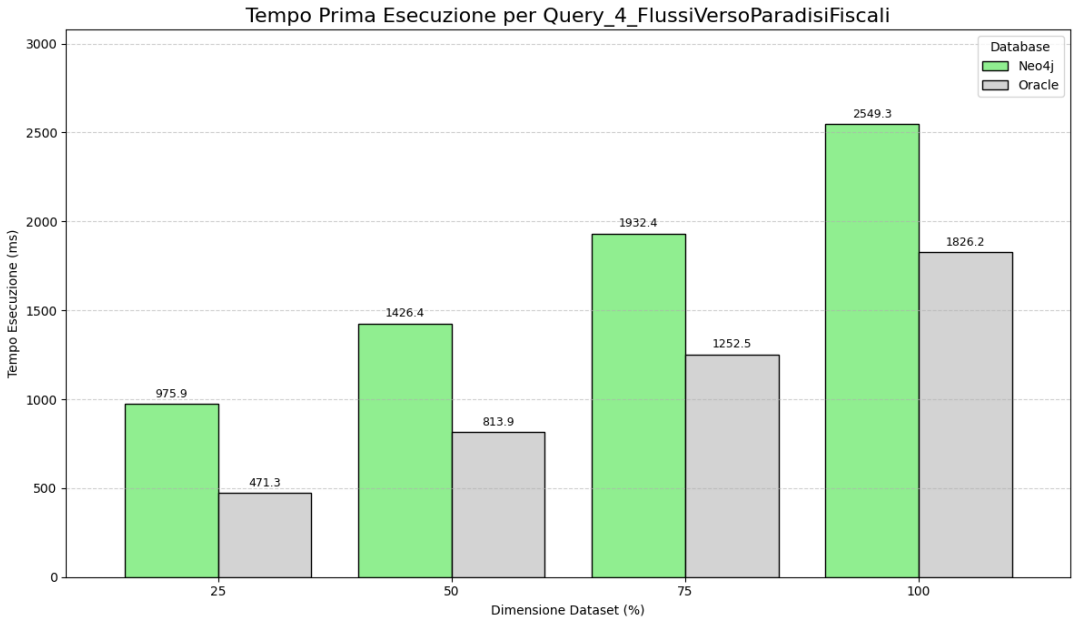

Figura 9: Q4 - Tempi di esecuzione a freddo.

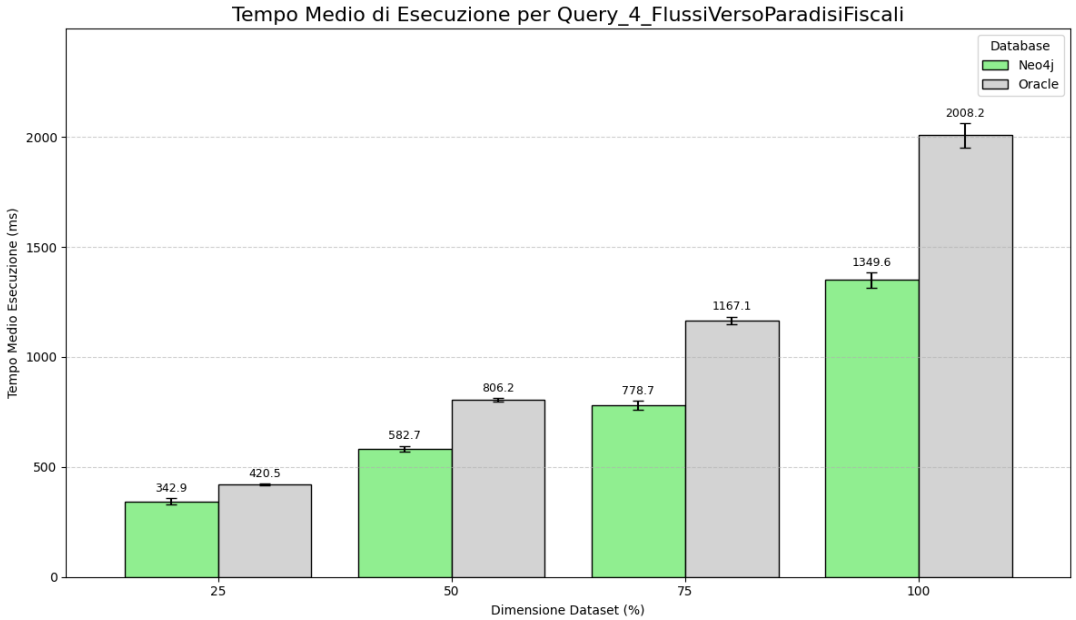

Figura 10: Q4 - Tempi medi di esecuzione a caldo.

Analisi Q4: Neo4j è di nuovo il vincitore assoluto. Questa query è una versione più complessa e realistica della Query 2. In SQL, si traduce in una catena di ben 5 `JOIN` che attraversa sei tabelle logiche. Il costo computazionale è molto elevato. La query Cypher, invece, è un capolavoro di espressività ed efficienza: descrive l'intero pattern e permette al planner di Neo4j di partire dai punti più selettivi (le nazioni con le proprietà richieste) e di esplorare i percorsi tra di essi, risultando ordini di grandezza più veloce.

6 Conclusioni Finali

I risultati sperimentali confermano in modo inequivocabile le ipotesi teoriche sui punti di forza e di debolezza dei due paradigmi di database. La scelta della tecnologia ottimale non è assoluta, ma strettamente dipendente dalla natura del carico di lavoro.

Oracle: il Re delle Operazioni Set-Based Oracle si è dimostrato imbattibile nelle query (1 e 3) la cui logica si basa sulla **scansione e filtraggio di grandi insiemi di dati con relazioni a bassa profondità**. Il suo motore SQL, affinato in decenni di sviluppo, è progettato per gestire in modo estremamente efficiente operazioni su tabelle. Quando la domanda è "trova tutti gli X che soddisfano la condizione Y", e Y richiede pochi `JOIN`, Oracle è la scelta naturale

Neo4j: il Maestro delle Connessioni Neo4j ha mostrato una superiorità schiacciante nelle query (2 e 4) focalizzate sull'**analisi di pattern e sull'attraversamento di relazioni complesse**. La sua architettura nativa a grafo permette di esplorare percorsi e vicinanze con una performance che non degrada con la dimensione totale del dataset. Quando la domanda è "trova i percorsi tra A e C che rispettano certe regole", Neo4j fornisce una risposta più rapida e con un linguaggio (Cypher) più espressivo.

Visione d'Insieme e Architetture Ibride Nessuno dei due database è una "pallottola d'argento". Un'analisi delle frodi completa e moderna richiede entrambe le capacità: l'analisi aggregata su grandi volumi e l'analisi approfondita dei pattern di connessione. Questo porta alla conclusione che l'approccio più efficace in un contesto enterprise non è scegliere l'uno *o* l'altro, ma utilizzarli entrambi in un'**architettura ibrida** (spesso definita *Polyglot Persistence*):

1. **Oracle** agirebbe come *System of Record* (SOR), il database transazionale (OLTP) che immagazzina in modo sicuro e affidabile tutti i dati anagrafici e finanziari.

2. **Neo4j** verrebbe utilizzato come un sistema analitico specializzato. I dati rilevanti verrebbero periodicamente estratti da Oracle e caricati in Neo4j per consentire agli analisti di eseguire query complesse sui pattern di frode, sfruttando la sua velocità e la visualizzazione intuitiva.

Questa sinergia permette di sfruttare il meglio di entrambi i mondi, combinando l'affidabilità e la potenza aggregata del mondo relazionale con la velocità e l'intuitività del mondo dei grafi per un'analisi delle frodi più efficace e completa.